

Consolidation Table (Fixed Groups)

Function-by-Function Code Walkthrough

A Jupyter-notebook-style deep dive into `src/visual.ts` — every function explained, with Python-style `'#'` comments annotating the TypeScript logic line by line, and flow diagrams for the key algorithms (run-length header grouping, the settings round-trip, and the overall render pipeline).

1. Overall render pipeline

Every Power BI custom visual has exactly one entry point that gets called again and again: **update()**. It runs whenever the data changes, a filter is applied, or the user touches anything in the Format pane. There is no persistent DOM — the function wipes the table and rebuilds it from scratch on every call. The diagram below traces that entire path for ConsolidationTableFixedGroups, from the very first line to the moment the table is mounted on the page.

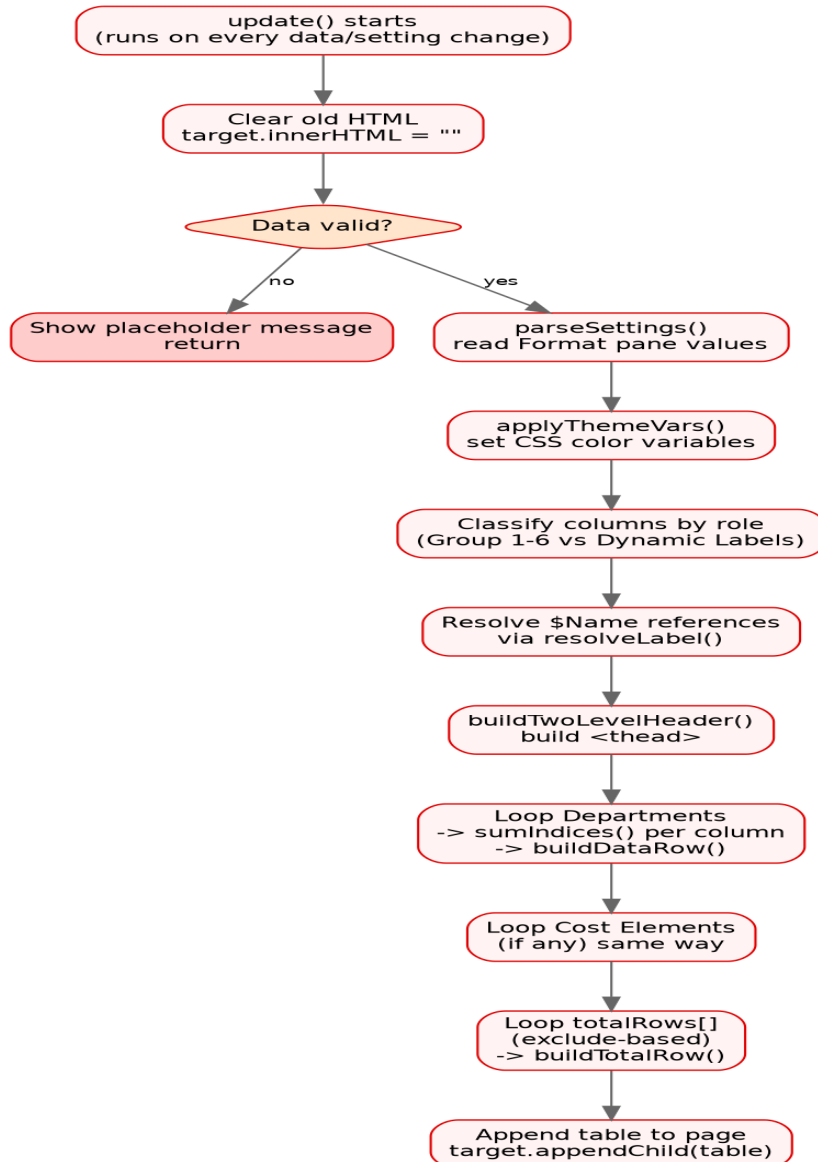


Figure 1 — the full update() render pipeline

2. Module-level helper functions

These are plain functions declared outside the class. They take input, return output, and never touch `this.settings` or the DOM directly — pure, easy-to-test building blocks used by the class methods later.

2.1 `getDefaultSettings()`

```
function getDefaultSettings(): ISettings
```

Returns a brand-new settings object with every property set to its factory default. Called once before the very first render, and again as a fallback whenever `dataView.metadata.objects` is missing.

```
# Python-style reading of the TypeScript body:
def get_default_settings():
    return {
        "negativeColor": "#E20000",      # red text for negative numbers
        "headerColor": "#E20000",        # red header background
        "groupHeaderColor": "#7F7F7F",    # grey department-row background
        "totalRowColor": "#F0F0F0",       # light grey total-row background
        "fontSize": 13,
        "defaultExpanded": True,          # departments start expanded
        "groupGapWidth": 8,               # px, white divider between groups
        "showToggleIcon": True,
        "boldMainRows": True,
        "boldChildNumbers": False,        # cost-element numbers NOT bold by default
        "groupLabels": ["", "", "", "", "", ""], # 6 empty bucket names
        "relabelMap": {},                 # no renames yet
        "totalRows": [                   # 3 optional custom total rows
            {"enabled": False, "label": "Total", "exclude": []},
            {"enabled": False, "label": "Total", "exclude": []},
            {"enabled": False, "label": "Total", "exclude": []},
        ],
        "descTwoLines": False,
        "descLine1": "Description",
        "descLine2": "",
        "descSeparatorSize": 1,
    }
```

Step-by-step:

- Nothing is read from Power BI here — every value is a hard-coded literal.
- The 3-item `totalRows` array is the reason the Format pane always shows exactly 3 configurable total-row slots, no more, no fewer.
- This function is the single source of truth for 'what does the visual look like the very first time it's dropped on a report, before anyone touches Format pane'.

2.2 `parseRelabelMap(text)`

```
function parseRelabelMap(text: string): Record<string, string>
```

Turns the single free-text 'Rename columns' box (e.g. "DB_PERFORMANCE=Perf;DB_FX=FX") into a lookup dictionary.

```
def parse_relabel_map(text):
    result = {} # empty dict to fill in
    if not text:
        return result # nothing typed -> nothing to parse
    for pair in text.split(";"): # split on semicolons first
        idx = pair.find("=") # find the FIRST '=' only
        if idx == -1:
            continue # malformed piece, e.g. no '=' -> skip it
        key = pair[:idx].strip() # everything before '=' -> the old name
        val = pair[idx+1:].strip() # everything after '=' -> the new name
        if key:
            result[key] = val # only store if key isn't blank
    return result
```

Step-by-step:

- Splitting is deliberately done on the FIRST '=' only (indexOf, not a naive split), so a replacement value that itself contains '=' won't corrupt the key.
 - Whitespace around both key and value is trimmed, so "A = B" and "A=B" behave identically.
 - Any malformed pair (missing '=') is silently skipped rather than crashing the visual.
-

2.3 parseDeptList(text)

```
function parseDeptList(text: string): string[]
```

Turns a comma-separated 'exclude list' (e.g. "Bad Debt, Regulatory") into an array of lower-cased, trimmed department names ready for case-insensitive matching.

```
def parse_dept_list(text):
    if not text:
        return [] # empty box -> empty list
    parts = text.split(",") # split on commas
    parts = [p.strip().lower() for p in parts] # trim + lowercase every piece
    parts = [p for p in parts if len(p) > 0] # drop empty pieces (extra commas)
    return parts
```

Step-by-step:

- Lower-casing happens HERE, once, rather than at comparison time later — so every later comparison is a cheap exact string match.
 - This is what powers the exclude-based Total rows: the visual compares this list against every department name (also lower-cased) to decide who's excluded.
-

2.4 relabel(map, key)

```
function relabel(map: Record<string,string>, key: string): string
```

One-line lookup: if the user renamed this key, return the new name; otherwise return the key unchanged.

```
def relabel(rename_map, key):
    # ternary: use the dictionary's value if it exists, else fall back to the key itself
    return rename_map[key] if key in rename_map else key
```

Step-by-step:

- Called for every single column's display name before it's drawn to the header.

2.5 resolveLabel(text, dynamicMap) - the '\$CurMonth' feature

```
function resolveLabel(text: string, dynamicMap: Record<string,string>): string
```

This is the function that makes a label like **\$CurMonth** resolve live to whatever value is currently in the 'Dynamic Labels' data bucket, instead of showing static text.

```
def resolve_label(text, dynamic_map):
    if text and text[0] == "$":
        key = text[1:].strip()
        if key in dynamic_map:
            return dynamic_map[key]
        else:
            return text
    return text
```

does the text start with a literal '\$' ?
drop the '\$', trim whitespace -> field name
found it -> return the LIVE value
not found -> return "\$CurMonth" as-is
(visible typo = easy to spot & fix)
no leading '\$' -> plain static text, unchanged

Step-by-step:

- The '\$' prefix is the entire trigger mechanism — anything not starting with '\$' is treated as ordinary literal text and passed through untouched.
- dynamic_map is built once per render inside update(), from whatever fields the user dropped into the 'Dynamic Labels' bucket (see section 3).
- Falling back to the raw '\$Name' text (instead of blanking it) is a deliberate debugging aid: a visible '\$Something' in the header means the name doesn't match anything in the bucket.

2.6 formatNumber(value, decimals)

```
function formatNumber(value: number, decimals = 0): string
```

Reproduces the classic accounting number format: thousands separators, and negative numbers shown in parentheses instead of with a minus sign.

```
def format_number(value, decimals=0):
    if value is None or is_nan(value):
        return ""
    abs_value = abs(value)
    formatted = f"{abs_value:,.{decimals}f}"
    if value < 0:
```

blank cell for missing data
strip the sign for now
thousands separator + fixed decimals

```
      return f"({formatted})"          # negative -> wrap in parentheses
s    return formatted                  # positive -> plain
```

Step-by-step:

- decimals defaults to 0 — every number in this visual is shown as a whole number unless a future change passes a different value.
 - This single function is called from every row-building function so the number format is guaranteed to be identical everywhere in the table.
-

3. Reading & classifying the DataView

This block lives inside `update()` itself (not a separate function), but it's dense enough to deserve its own walkthrough. Its job: take the flat list of columns Power BI hands over, and sort each one into either 'a Group 1-6 data column' or 'a Dynamic Label source', while simultaneously harvesting the live values for any Dynamic Label fields.

```
rawColumns = [] # will hold every REAL data column (Group 1-6)
dynamic_label_values = {} # will hold {fieldName: liveValue} for $-references

for col in categorical.values: # loop every column Power BI gave us
    roles = col.source.roles or {} # which Data-pane role(s) is it bound to?

    if roles.get("dynamicLabels"): # -- CASE 1: it's a Dynamic Label field --
        first_val = next((v for v in col.values if v is not None), None)
        dynamic_label_values[col.source.displayName] = str(first_val) if first_val else ""
        continue # do NOT add it as a visible column

    group_index = -1 # -- CASE 2: look for Group 1-6 role --
    for g in range(6):
        if roles.get(GROUP_ROLES[g]):
            group_index = g
            break
    if group_index == -1:
        continue # bound to neither -> ignore entirely

    rawColumns.append({"column": col, "groupIndex": group_index, "subLabel": ""})

# ---- second pass: NOW that dynamic_label_values is fully populated, resolve names ----
for c in rawColumns:
    renamed = relabel(settings.relabelMap, c["column"].source.displayName)
    c["subLabel"] = resolve_label(renamed, dynamic_label_values)
```

Why two separate passes? The Dynamic Labels bucket might be listed AFTER a Group column in Power BI's column order. If label-resolving happened inside the same loop, a `$CurMonth` reference could be resolved before `CurMonth`'s value had even been read yet. Splitting into 'collect everything first' then 'resolve names second' guarantees `dynamic_label_values` is complete before it's ever read from.

3.1 Computing `isGroupStart`

```
columns = []
for idx, c in enumerate(rawColumns):
    is_group_start = (idx == 0) or (rawColumns[idx-1]["groupIndex"] != c["groupIndex"])
    columns.append(**c, "isGroupStart": is_group_start)
    #          ^^^ spread operator: copy every existing field, then add one more
```

A column is flagged as 'the start of a new group' either if it's literally the first column in the table, or if the group index of the column right before it is different. This single boolean is what later triggers the white vertical divider (CSS class `ct-group-gap`) between Group 1's columns and Group 2's columns.

4. Row iteration & aggregation

4.1 sumIndices(col, indices)

```
private sumIndices(col: DataViewValueColumn, indices: number[]): number
```

The single function responsible for every numeric total in the whole visual — department rows, cost-element rows, and custom total rows all funnel through this.

```
def sum_indices(col, indices):
    total = 0
    for i in indices:
        v = col.values[i]
        if v is not None and not is_nan(v):
            total += v
    return total
```

Step-by-step:

- 'indices' is a plain list of row numbers — for a Department row it's every row belonging to that department; for a Total row it's every row NOT in the exclude list.
- Because the SAME function handles both cases, department totals and custom Total rows are mathematically guaranteed to be consistent — there's only one place a bug could hide.

4.2 The Department to Cost Element loop

```
dept_order = []
for d in dept_values:
    if d not in dept_order:
        dept_order.append(d)

for dept in dept_order:
    if dept not in expanded_state:
        expanded_state[dept] = settings.defaultExpanded

    dept_indices = [i for i in all_indices if dept_values[i] == dept]
    has_children = ce_values is not None

    dept_totals = [sum_indices(c["column"], dept_indices) for c in columns]
    tbody.append(build_data_row(dept, dept_totals, columns,
                                is_parent=True, clickable=has_children, parent_key=dept))

    if has_children:
        ce_order = []
        for i in dept_indices:
            ce = ce_values[i]
            if ce not in ce_order:
                ce_order.append(ce)

        for ce in ce_order:
```



```

ce_indices = [i for i in dept_indices if ce_values[i] == ce]
ce_totals = [sum_indices(c["column"], ce_indices) for c in columns]
row = build_data_row(ce, ce_totals, columns,
                    is_parent=False, clickable=False, parent_key=dept)
row.dataset["parent"] = dept          # tag: "I belong to this department"
row.style.display = "" if expanded_state[dept] else "none"
tbody.append(row)

```

The nested structure mirrors the visual output exactly: outer loop = one row per Department (always visible, always aggregated across every cost element beneath it); inner loop = one row per Cost Element (only rendered if a Cost Element field was dropped at all, and hidden via inline style.display when the parent is collapsed).

5. Header construction

The two-row header (group names on top, individual column names underneath) is built by walking the already-sorted 'columns' array once, and using a classic 'run-length grouping' technique to merge consecutive columns from the same bucket into a single wide `<td colSpan=N>` cell.

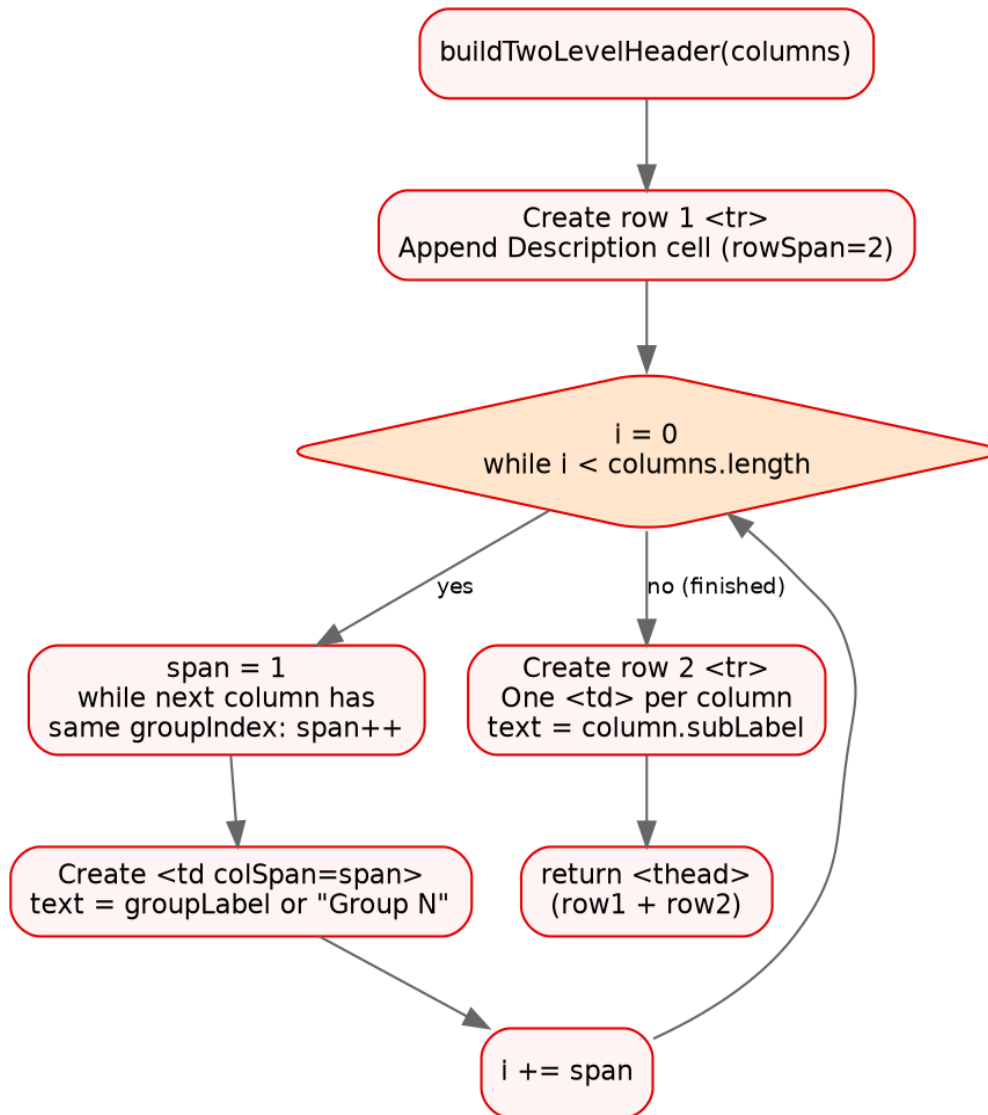


Figure 2 — `buildTwoLevelHeader()` run-length grouping loop

5.1 buildDescriptionCell()

```
private buildDescriptionCell(): HTMLTableCellElement
```

Builds the single top-left header cell (spanning both header rows), either as one line of text or two lines separated by a thin divider, per the 'Description header cell' Format pane settings.

```
def build_description_cell():
    cell = create_element("td")
    cell.rowSpan = 2
    cell.className = "ct-desc-header"
    # always spans both header rows
```

```

if settings.descTwoLines:
    cell.append(text_node(resolve_label(settings.descLine1, dynamic_label_values)))
    sep = create_element("div")
    sep.className = "ct-desc-separator"
    sep.style.borderTopWidth = f"{settings.descSeparatorSize}px"
    cell.append(sep)
    cell.append(text_node(resolve_label(settings.descLine2, dynamic_label_values)))
else:
    cell.textContent = resolve_label(settings.descLine1, dynamic_label_values)

return cell

```

Step-by-step:

- Both lines pass through resolveLabel() too, so \$CurMonth also works inside the Description cell, not just column headers.

5.2 buildTwoLevelHeader(columns)

private buildTwoLevelHeader(columns: IColumnInfo[]): HTMLTableSectionElement

Assembles the <thead> with exactly two <tr> rows using the run-length grouping algorithm shown in Figure 2.

```

def build_two_level_header(columns):
    thead = create_element("thead")

    # ----- ROW 1: group names, colSpan-merged -----
    top_row = create_element("tr")
    top_row.append(build_description_cell())

    i = 0
    while i < len(columns):
        group_idx = columns[i]["groupIndex"]
        span = 1
        # grow the span while the NEXT column belongs to the same group
        while i + span < len(columns) and columns[i+span]["groupIndex"] == group_idx:
            span += 1

        cell = create_element("td")
        cell.colSpan = span
        if i > 0:
            cell.classList.add("ct-group-gap")          # white divider before this gro
up
        label = resolve_label(settings.groupLabels[group_idx], dynamic_label_values)
        cell.textContent = label if label else f"Group {group_idx + 1}"
        top_row.append(cell)

        i += span                                     # JUMP by span, not by 1!

    thead.append(top_row)

    # ----- ROW 2: one plain cell per column -----
    sub_row = create_element("tr")
    for idx, c in enumerate(columns):
        cell = create_element("td")

```

```
        if c["isGroupStart"] and idx > 0:
            cell.classList.add("ct-group-gap")
            cell.textContent = c["subLabel"]
            sub_row.append(cell)
        thead.append(sub_row)

    return thead
```

Step-by-step:

- The key trick is 'i += span' instead of 'i += 1' — once a group's whole width is known, the outer loop jumps straight past it to the next group's first column.
 - Row 2 needs none of that span logic — it's a flat one-cell-per-column row, using the isGroupStart flag only to decide where to draw the divider border.
-

6. Expand/collapse interaction & row rendering

6.1 appendToggleAndLabel(cell, label, clickable, parentKey)

```
private appendToggleAndLabel(cell, label: string, clickable: boolean, parentKey:
string)
```

Adds the optional +/- icon and the click handler that expands/collapses a department's cost-element rows. Shared by every clickable department row.

```
def append_toggle_and_label(cell, label, clickable, parent_key):
    if clickable:
        if settings.showToggleIcon:
            icon = create_element("span")
            icon.className = "ct-toggle-icon"
            icon.textContent = "- " if expanded_state[parent_key] else "+ "
            cell.append(icon)
            cell._toggle_icon = icon          # stash a reference for fast re-use late
r
        cell.style.cursor = "pointer"

        def on_click():
            expanded_state[parent_key] = not expanded_state[parent_key]    # flip the fl
ag
            # find every row tagged as a child of this department...
            child_rows = query_all(f'tr[data-parent="{css_escape(parent_key)}"]')
            for r in child_rows:
                r.style.display = "" if expanded_state[parent_key] else "none"
            # ...and refresh the icon glyph to match the new state
            if cell._toggle_icon:
                cell._toggle_icon.textContent = "- " if expanded_state[parent_key] els
e "+ "

            cell.add_event_listener("click", on_click)

        cell.append(text_node(label))
```

Step-by-step:

- The icon is entirely optional (showToggleIcon setting) but the click handler is always attached when clickable=True — so the row stays expandable even with the icon hidden.
- CSS.escape() guards against a department name containing quote characters that would otherwise break the [data-parent="..."] selector string.

6.2 buildDataRow(...) and buildTotalRow(...)

```
private buildDataRow(label, values, columns, isParent, clickable, parentKey)
```

Renders one <tr> of numeric data — used for both department rows and cost-element rows, distinguished by the isParent flag.

```

def build_data_row(label, values, columns, is_parent, clickable, parent_key):
    row = create_element("tr")
    row.className = "ct-dept-row" if is_parent else "ct-ce-row"

    label_cell = create_element("td")
    label_cell.className = "ct-dept-title" if is_parent else "ct-ce-title"
    if is_parent:
        append_toggle_and_label(label_cell, label, clickable, parent_key)
        label_cell.style.fontWeight = "bold" if settings.boldMainRows else "normal"
    else:
        label_cell.textContent = label # cost-element name: always plain
    row.append(label_cell)

    for idx, v in enumerate(values):
        cell = create_element("td")
        cell.className = "ct-value-cell"
        if columns[idx]["isGroupStart"] and idx > 0:
            cell.classList.add("ct-group-gap")
        if v < 0:
            cell.style.color = settings.negativeColor
        if is_parent:
            cell.style.fontWeight = "bold" if settings.boldMainRows else "normal"
        else:
            cell.style.fontWeight = "bold" if settings.boldChildNumbers else "normal"
        cell.textContent = format_number(v, 0)
        row.append(cell)

    return row

```

Step-by-step:

- `boldMainRows` controls Department rows' font weight (label AND numbers together).
 - `boldChildNumbers` is INDEPENDENT: it controls only the numeric cells of Cost Element rows, while the cost-element NAME (`label_cell`) is always plain — this is the exact mechanism behind the 'Bold numbers for Cost Element rows (name stays normal)' toggle.
-

7. The settings round-trip

Two functions form a mirror pair: `parseSettings()` reads whatever the user configured in the Format pane, and `enumerateObjectInstances()` tells Power BI what to display the NEXT time that same pane is opened. Get one wrong and the pane either ignores your changes, or resets to defaults every time you reopen it.

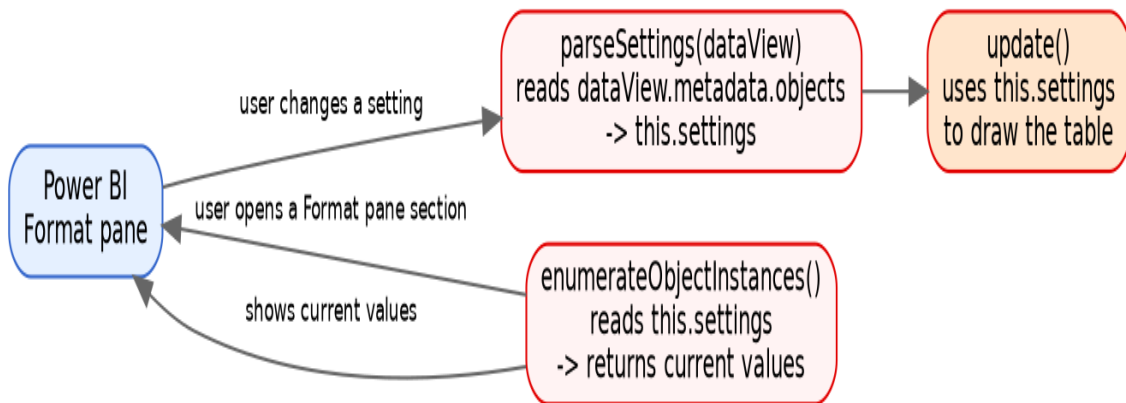


Figure 3 — `parseSettings()` and `enumerateObjectInstances()` are mirror images of each other

7.1 `parseSettings(dataView)`

```
private parseSettings(dataView: DataView): ISettings
```

Reads `dataView.metadata.objects` (where Power BI stores every Format-pane change) and builds a fully-populated `ISettings` object, falling back to defaults for anything the user hasn't touched yet.

```
def parse_settings(data_view):
    settings = get_default_settings()                # start from factory defaults
    objects = data_view.metadata.objects if data_view and data_view.metadata else None
    if not objects:
        return settings                               # nothing customized -> defaults

    dp = objects.get("dataPoint")
    tf = objects.get("tableFormat")
    gl = objects.get("groupLabels")
    cl = objects.get("columnLabels")
    tr = objects.get("totalRows")
    dh = objects.get("descriptionHeader")

    if dp:
        if dp.negativeColor and dp.negativeColor.solid:
            settings.negativeColor = dp.negativeColor.solid.color
        # ... same double-checked pattern repeats for every color

    if gl:
        # one line replaces what would otherwise be 6 near-identical if-statements
        settings.groupLabels = [
            gl.get(f"group{n}Label", "") if isinstance(gl.get(f"group{n}Label"), str) else ""
            for n in range(1, 7)
        ]
```

```

if tr:
    settings.totalRows = [
        {
            "enabled": tr.get(f"enableRow{n}", False),
            "label": tr.get(f"label{n}") or "Total",
            "exclude": parse_dept_list(tr.get(f"excludeRows{n}", "")),
        }
        for n in range(1, 4)
    ]

# ... dh (descriptionHeader) parsed the same defensive way

return settings

```

Step-by-step:

- Every single read is guarded by a type check (`typeof x === '...'`) because Power BI hands these values back as loosely-typed 'any' — a missing or malformed property must NOT crash the whole render.
- The pattern 'start from defaults, then overwrite only what's actually present' means a partially-configured visual (some settings touched, others not) never ends up in a broken, half-initialized state.

7.2 enumerateObjectInstances(options)

`public enumerateObjectInstances(options): VisualObjectInstanceEnumeration`

Called by Power BI every time the user opens a Format pane section, asking 'what values should the controls in this section currently show?'

```

def enumerate_object_instances(options):
    object_name = options.objectName
    settings = self.settings or get_default_settings()
    instances = []

    if object_name == "dataPoint":
        instances.append({
            "objectName": object_name,
            "properties": {
                "negativeColor": {"solid": {"color": settings.negativeColor}},
                "headerColor": {"solid": {"color": settings.headerColor}},
                "groupHeaderColor": {"solid": {"color": settings.groupHeaderColor}},
                "totalRowColor": {"solid": {"color": settings.totalRowColor}},
            },
            "selector": None,
        })

    if object_name == "totalRows":
        props = {}
        for idx, tr in enumerate(settings.totalRows):
            n = idx + 1
            props[f"enableRow{n}"] = tr.enabled
            props[f"label{n}"] = tr.label
            props[f"excludeRows{n}"] = ", ".join(tr.exclude) # array -> comma text

```



```
        instances.append({"objectName": object_name, "properties": props, "selector": None})

        # ... identical pattern repeats for tableFormat, groupLabels, columnLabels, descriptionHeader

    return instances
```

Step-by-step:

- `tr.join(tr.exclude)` is the exact inverse of `parseDeptList()` — together they form a lossless round-trip: text box → array → (later) text box again.
- If this function returned the DEFAULT settings instead of `self.settings`, every Format pane section would visually 'forget' the user's changes the moment it was reopened, even though the visual itself kept rendering correctly.

End of walkthrough. Every function in `src/visual.ts` has now been covered: the module-level helpers, the data classification and aggregation logic inside `update()`, the header run-length grouping algorithm, row/expand-collapse rendering, and the settings round-trip.